KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions"

Submitted as a part of the course

AGENTIC AI

Submitted By

Student Name	Enrollment No.
_____ _	_____

Program: B.Tech CSE / B.Tech CSE-AIML
Semester: VI
Section: _____

Under the Guidance of

Faculty Name
Assistant Professor
Department of Computer Science & Information Technology

Karnavati University, Gandhinagar

Academic Year: 2026–27

CERTIFICATE

This is to certify that the mini project entitled **“AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions”** has been successfully completed by the following students of B.Tech AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions Semester VI, Karnavati University, as a part of the course **Agentic AI** during the academic year 2026–27.

Sr. No.	Student Name	Enrollment No.
1		
2		
3		
4		

The project has been carried out under the guidance and supervision of the faculty member. The work presented in this report is the original work carried out by the students as part of their mini project.

Faculty
Name:
Signature:
Date: _____

Guide
_____
_____

DECLARATION

We hereby declare that the mini project entitled **"AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions"** submitted as part of the course **Agentic AI** is our original work carried out under the guidance of the faculty member.

We further declare that this work has not been submitted previously to any other institution or university for the award of any degree, diploma, or academic certification.

All research papers, datasets, frameworks, APIs, libraries, websites, and other resources used during the development of the project have been appropriately acknowledged and referenced.

Sr. No.	Student Name	Enrollment No.
1		
2		
3		
4		

Date: _____

Place: Gandhinagar

ACKNOWLEDGEMENT

We thank our faculty guide for the direction, feedback and patience given throughout this mini project, and the Department of Computer Science & Information Technology, Unitedworld Institute of Technology, Karnavati University, for including Agentic AI in the curriculum and for the laboratory and computing facilities that made the work possible.

The simulation work builds on the disaster-zone world and the Hexa-X airframe developed by our team, and on open-source software from the ArduPilot, Gazebo and Python communities: ArduPilot SITL, MAVProxy and pymavlink, Gazebo Harmonic, Streamlit, Plotly, pandas, Pydantic and Ollama. We acknowledge the authors of the third-party simulation models listed in the references.

Finally, we thank every team member for their contribution to the design, implementation, testing and documentation of AeroMind.

ABSTRACT

After a drone flight, the information needed to understand what happened is scattered across a telemetry log, a mission file, a parameter file and the pilot's memory, and reading it takes expertise. Most operators therefore never learn why a flight drifted, ran its battery low or lost its telemetry link, and repeat the same mistakes. This project presents **AeroMind**, a multi-agent Agentic AI system that analyses the files of one ArduPilot flight and produces a one-page verdict, a phase-by-phase story, a telemetry-link timeline, root causes, a next-mission risk score and a corrected mission file.

Deterministic Python tools convert MAVLink logs into verified metrics with Good/Caution/Bad statuses. An orchestrator builds a manifest, obtains a plan from a large language model (LLM), checks the plan against rules, and runs five specialist agents (Flight Summary, Telemetry Link, Diagnosis, Risk and Advisor) on a shared state. A deterministic Verifier rejects any claim not supported by the data and triggers one self-correction round. A flight history provides long-term memory: personal baselines, battery prediction and recurring issues. The system is built in Python with Pydantic, Streamlit and Plotly, and runs with Claude through the Anthropic API or a local Qwen2.5-3B model through Ollama.

Eight flights were flown in ArduPilot SITL in a Gazebo disaster world with injected faults and known ground truth. AeroMind's agents detected all four injected faults with the correct root cause, located both telemetry dropouts within 0.1 s of the truth, raised no false alarm on the baseline flight and ranked next-mission risk correctly. The mission they corrected for the undersized-battery scenario raised the battery at landing from 14 % to 25 %, and the speed fix reduced the wind-scenario cross-track error from 1.8 m to 1.44 m. The work shows that LLM agents paired with deterministic verification produce useful, grounded flight analysis even with a small model running locally.

TABLE OF CONTENTS

ACKNOWLEDGEMENT....4
ABSTRACT....5
LIST OF FIGURES....8
LIST OF TABLES....9
CHAPTER 1 – INTRODUCTION....10
1.1 Introduction....10
1.2 Background....10
1.3 Problem Statement....10
1.4 Motivation....11
1.5 Objectives....11
1.6 Scope of the Project....11
CHAPTER 2 – LITERATURE REVIEW....12
2.1 Existing Systems and Research....12
2.2 Comparative Analysis....12
2.3 Research Gap....12
CHAPTER 3 – PROPOSED SYSTEM....13
3.1 Proposed Solution....13
3.2 System Requirements....13
3.3 Functional Requirements....14
3.4 Non-Functional Requirements....14
CHAPTER 4 – AGENT ARCHITECTURE....15
4.1 Overall System Architecture....15
4.2 Architecture Components....15
4.3 Agent Roles and Responsibilities....15
4.4 Agent Workflow....16
CHAPTER 5 – AGENTIC AI CONCEPTS USED....17
5.1 ReAct....17
5.2 Planning....17
5.3 Tool Calling....17
5.4 Memory....18
5.5 Retrieval-Augmented Generation....18
5.6 Multi-Agent System....18
5.7 Human-in-the-Loop....18
CHAPTER 6 – METHODOLOGY....19
6.1 Input Processing....19
6.2 Task Understanding....19
6.3 Task Planning....19
6.4 Reasoning and Decision Making....19

6.5 Tool Selection....19
6.6 Task Execution....19
6.7 Observation and Feedback....19
6.8 Final Response Generation....19
CHAPTER 7 – IMPLEMENTATION....20
7.1 Project Structure....20
7.2 Agent Implementation....20
7.3 Tool Implementation....20
7.4 Memory Implementation....21
7.5 RAG Implementation....21
7.6 User Interface....21
CHAPTER 8 – RESULTS AND OUTPUT....22
8.1 Test Cases....22
8.2 Sample Outputs....23
8.3 Agent Execution Trace....24
CHAPTER 9 – PERFORMANCE EVALUATION....25
CHAPTER 10 – CHALLENGES AND LIMITATIONS....26
CHAPTER 11 – FUTURE SCOPE....27
CHAPTER 12 – CONCLUSION....28
REFERENCES....29
APPENDIX....30
Appendix A – Source Code....30
Appendix B – Additional Screenshots....30
Appendix C – Sample Prompts....36
Appendix D – Test Data....36

LIST OF FIGURES

Figure 1: Overall system architecture of AeroMind.....15
Figure 2: Agent workflow of one analysis.....16
Figure 3: Planning and execution flow of the orchestrator.....17
Figure 4: Dashboard: input form, Summary and Agents tabs.....21
Figure 5: The Gazebo test world and the Hexa-X during a SITL mission.....22
Figure 6: Generated report.html for S2.....24
Figure 7: Ask tab (S3): an answer verified against the cited metrics.....30
Figure 8: Track tab (S3): planned route, flown path coloured by link quality.....31
Figure 9: Telemetry tab (S3): link strip, message rate and dropout table.....32
Figure 10: History tab (S2 fixed): baseline, trends and what worked before.....33
Figure 11: Power & attitude tab (S2).....34
Figure 12: More views of the simulation world.....35
Figure 13: Hexa-X over the collapsed buildings.....35
Figure 14: Hexa-X flying past the flood zone.....35

LIST OF TABLES

Table 1: Literature review summary.....12
Table 2: Comparison of existing tools and the proposed system.....12
Table 3: Hardware requirements.....13
Table 4: Software requirements.....13
Table 5: Architecture components.....15
Table 6: Agent roles and responsibilities.....16
Table 7: Tools: purpose, input and output.....18
Table 8: Tool implementation.....21
Table 9: Test scenarios.....22
Table 10: Test cases and actual results.....23
Table 11: Performance evaluation.....25
Table 12: Results before and after the fixed missions.....25
Table 13: Sample prompts.....36
Table 14: Test data.....36

CHAPTER 1 – INTRODUCTION

1.1 Introduction

Artificial Intelligence (AI) builds systems that perceive, reason and decide. **Generative AI** produces new content: large language models (LLMs) such as Claude and Qwen generate text, code or structured data from a prompt. **Agentic AI** uses an LLM as the decision-making core of a system that pursues a goal over several steps: it plans, acts through tools or other agents, observes the results and adjusts [1], [11]. A chatbot answers one message at a time; an agent works towards a goal. AeroMind shows each property of an agent:

- **Autonomy:** once a flight's files are dropped in, the whole analysis runs without further input.
- **Planning:** an orchestrator decides which agents to run, in what order and why.
- **Reasoning:** each agent explains what happened and why, citing the metrics it relies on.
- **Tool usage:** Python tools parse the logs, compute metrics, predict battery use and edit the mission.
- **Memory:** every analysed flight is stored and later flights are compared with the pilot's own baseline.
- **Goal-oriented execution:** work continues, with a self-correction round, until the result is verified.

1.2 Background

ArduPilot drones record MAVLink telemetry, saved by ground stations as a `.tlog` file [8], [9]. Tools such as UAV Log Viewer [10] and PX4 Flight Review [12] plot it but leave interpretation to an expert. AeroMind gives the arithmetic to tools and the judgement to agents, and checks the agents against the tools.

1.3 Problem Statement

The objective of this project is to develop an Agentic AI-based system that analyses the telemetry log, mission and parameters of an ArduPilot flight, explains how the flight and its telemetry link went, identifies what went wrong and why, scores the risk of the next mission and produces a corrected mission file. Existing systems plot the data but leave diagnosis, risk

assessment and mission correction to an expert. Therefore, there is a need for a system that can reason over flight data like an expert while every statement it makes is checked against the data.

1.4 Motivation

Our team builds and flies drones and spent hours working out from logs why a flight behaved as it did; small teams rarely have a flight-test engineer. Plotting tools leave interpretation to the user, and a stand-alone LLM can state numbers that are not in the data [6]. Agentic AI lets planned, specialised agents explain and advise while tools keep every number exact.

1.5 Objectives

1. To design a multi-agent system that turns one flight's files into a verdict, a flight story, a telemetry-link analysis, root causes, a next-mission risk score and a corrected mission.
2. To implement LLM-planned orchestration of specialist agents over a shared state, with rule-checked plans.
3. To integrate deterministic tools for log ingestion, metrics, risk scoring and bounded mission editing.
4. To implement long-term memory of flights for baselines, battery prediction and recurring issues.
5. To verify every agent claim against the data and evaluate the system on flights with known injected faults.

1.6 Scope of the Project

- **Target users:** pilots and student teams flying ArduPilot multirotors.
- **Supported tasks:** post-flight analysis of `.tlog` files (several per flight are merged) with an optional mission, parameters and pilot notes; questions about the flight; a corrected mission file.
- **Technologies:** Python, Claude or Qwen2.5-3B agents, Streamlit; SITL and Gazebo for testing.
- **Usage and limitations:** after each flight; `.tlog` logs; tested on simulated flights.

CHAPTER 2 – LITERATURE REVIEW

2.1 Existing Systems and Research

Table 1 summarises the work that shaped AeroMind.

Table 1: Literature review summary

Sr. No.	Paper/System	Year	Approach	Key Finding	Limitation
1	ReAct [1]	2023	Reasoning traces with tool actions	Reasoning and acting beat either alone	Errors propagate
2	AutoGen [2]	2023	Conversing specialist agents	Tasks decompose well across agents	No check against data
3	MetaGPT [3]	2024	Role agents, structured hand-offs	Structure reduces errors	Software engineering only
4	Reflexion [4], Self-Refine [5]	2023	Self-critique and retry	Feedback improves results	Critic can accept wrong answers
5	Log viewers [9], [10], [12]	–	Plot flight logs	Reliable plots of every signal	Interpretation left to user

2.2 Comparative Analysis

Table 2: Comparison of existing tools and the proposed system

Feature	Existing System (log viewers)	Proposed System (AeroMind)
Autonomous Execution	No: the user reads each plot	Yes: files in, full analysis out
Task Planning	No	Orchestrator plans the agents; plan checked by rules
Reasoning	No	Agents explain causes and risk, citing metrics
Tool Usage	Plotting only	Ingest, metrics, memory, risk rules, mission editing
Memory	No	Flight history: baselines, battery prediction, recurring issues
RAG	No	Not used
Multi-Agent Support	No	Five specialist agents and a Verifier
Human Intervention	Entirely manual	Pilot reviews the fixed mission before flying

2.3 Research Gap

As Table 2 shows, log tools are exact but explain nothing, while agent frameworks [1]–[5] do not check what an agent says against data [6]. AeroMind combines both with a Verifier.

CHAPTER 3 – PROPOSED SYSTEM

3.1 Proposed Solution

- **User interaction:** the user uploads a flight's files in a Streamlit dashboard or `run.py`, and can ask questions in the Ask tab.
- **Agents and LLM:** five specialist agents (Flight Summary, Telemetry Link, Diagnosis, Risk, Advisor) run on Claude [14] or a local Qwen2.5-3B model [13] through Ollama [15].
- **Planning and reasoning:** an orchestrator plans which agents run; each agent reasons over verified metrics.
- **Tools and memory:** Python tools compute the metrics and edit the mission; a flight history is the long-term memory. No knowledge base or retrieval is used.
- **Execution and final response:** a Verifier checks every claim; the approved result is shown and saved as a report, a fixed mission and a change log.

3.2 System Requirements

3.2.1 Hardware Requirements

Table 3: Hardware requirements

Component	Requirement
Processor	AMD Ryzen 5 7535HS, 6 cores / 12 threads
RAM	16 GB (analysis alone needs under 1 GB)
Storage	About 25 MB; 3.3 GB more for the optional local LLM
GPU / Internet	NVIDIA GeForce RTX 2050, 4 GB / for the Claude API (both optional)

3.2.2 Software Requirements

Table 3 lists the hardware used and Table 4 the software.

Table 4: Software requirements

Software/Technology	Version/Specification
Operating System	Ubuntu 22.04.5 LTS
Programming Language	Python 3.10.12 (Lua for the simulated battery)
Agent Framework	Own orchestrator in Python; Pydantic 2.13 schemas [19]
LLM	Claude (Anthropic SDK 2.1.0) or Qwen2.5-3B via Ollama 0.40.8
Frontend	Streamlit 1.57 [20], Plotly 6.7 [21], Jinja2 3.1
Data and parsing	pymavlink 2.4.49 [16], pandas 2.3 [22], NumPy 2.2
Simulation	ArduCopter SITL, MAVProxy 1.8.74, Gazebo Harmonic 8.15 [18]

3.3 Functional Requirements

1. Accept one or more `.tlog` files with an optional mission, parameters and pilot notes.
2. Understand the flight: compute metrics with Good/Caution/Bad statuses and locate dropouts.
3. Generate a plan of agents and run them, using the tools' output.
4. Verify every agent claim and maintain a memory of past flights.
5. Generate the final response: verdict, causes, risk, advice and a corrected mission.
6. Handle unsuccessful LLM calls by retrying, then using a rule-based answer.

3.4 Non-Functional Requirements

- **Accuracy:** no number reaches the user unchecked. **Reliability:** every analysis completes.
- **Performance:** the tool stage takes under 1 s; total time depends on LLM inference.
- **Scalability:** flights are independent runs; the history grows by one record per flight.
- **Security:** validated uploads, escaped text, local-only dashboard, API keys kept out of the code.
- **Usability and maintainability:** one-page summary; small typed modules with 247 automated tests.

CHAPTER 4 – AGENT ARCHITECTURE

4.1 Overall System Architecture

Figure 1 shows the architecture as implemented: inputs pass through Python tools into a shared state, the orchestrator plans and runs the agents, and the Verifier lets only approved output reach the user and the flight history.

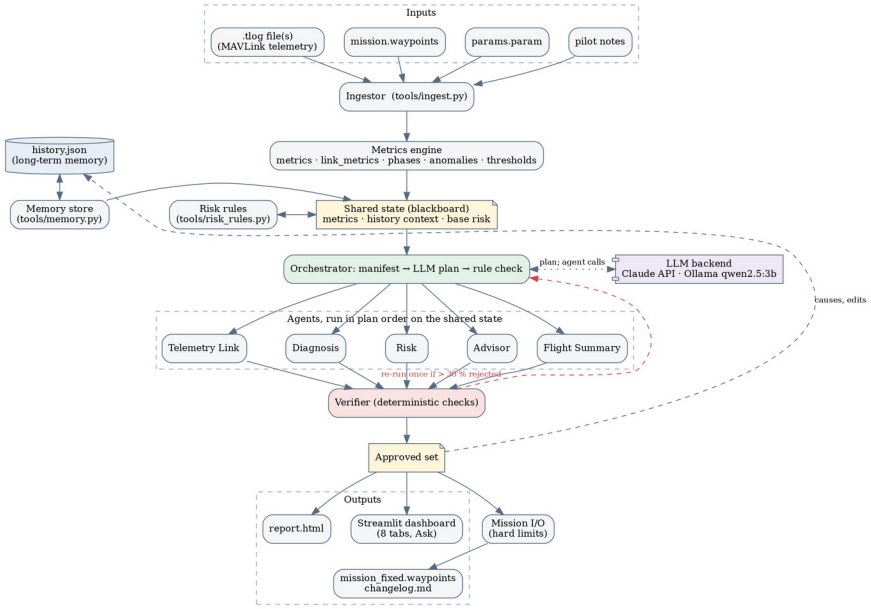


Figure 1: Overall system architecture of AeroMind

4.2 Architecture Components

Table 5: Architecture components

Component	Description
User Interface	Streamlit dashboard `app.py` (tabs, Ask chat); command line `run.py`
LLM	`agents/llm.py`: Claude (structured output) or Ollama (JSON schema)
Agents	Five agents, each a prompt and a Pydantic schema, run by `run_agent()`
Planner / Executor	`orchestrator.py`: manifest → LLM plan → rule check → run → verify
Tools	`tools/`: ingest, metrics, link metrics, risk rules, memory, mission I/O, report
Memory	`history.json` (long-term) and the shared state of one analysis
Output Generator	`pipeline.py`: state.json, report.html, fixed mission, change log

4.3 Agent Roles and Responsibilities

Table 5 describes the components and Table 6 the agents implemented.

Table 6: Agent roles and responsibilities

Agent	Role	Responsibility
Orchestrator	Planner, executor	Plans, checks the plan, runs agents, triggers refinement
Flight Summary	Narrator	Verdict, a line per flight phase, 3-sentence summary
Telemetry Link	Link analyst	Uptime, dropouts (time, duration, waypoint), packet loss
Diagnosis	Analyst	Findings citing metrics; up to 3 root causes with confidence
Risk	Assessor	Adjusts the rule-based risk score by at most ±10 with a reason
Advisor	Mission planner	Mission edits tied to causes; top 3 improvements
Verifier	Reviewer	Checks every claim and number against the data

4.4 Agent Workflow

1. The user provides the flight's files (and optional notes).
2. Tools compute the metrics, statuses and memory context (objective understood).
3. The orchestrator creates the plan of agents.
4. Each agent receives the tools' output it needs.
5. The agents run in order on the shared state.
6. The Verifier observes every output.
7. Agents with over 30 % of claims rejected are identified.
8. Those agents refine their answer once with the reasons.
9. The approved result is saved and presented.

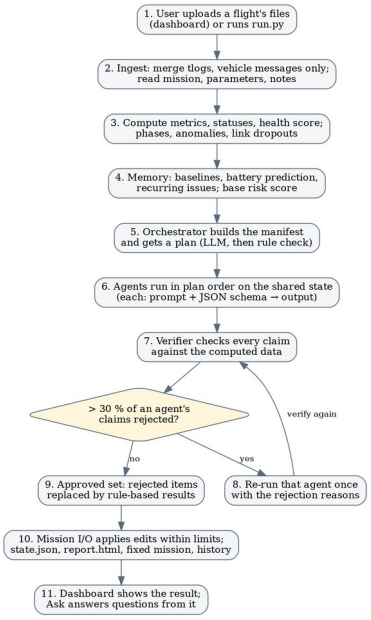


Figure 2: Agent workflow of one analysis

Figure 2 shows these steps.

CHAPTER 5 – AGENTIC AI CONCEPTS USED

5.1 ReAct

The tools always run first so none can be skipped [1]. The reason → act → observe cycle appears as verify and refine: the agent reasons and answers, the Verifier observes, and the agent acts again on the feedback.

5.2 Planning

Plan-and-execute with dynamic planning (Figure 3): the LLM receives a manifest (inputs, statuses, problems, history) and returns an ordered list of agents with reasons; `checked_plan()` enforces the rules (Flight Summary first, Verifier last, Diagnosis only when something is Caution or Bad).

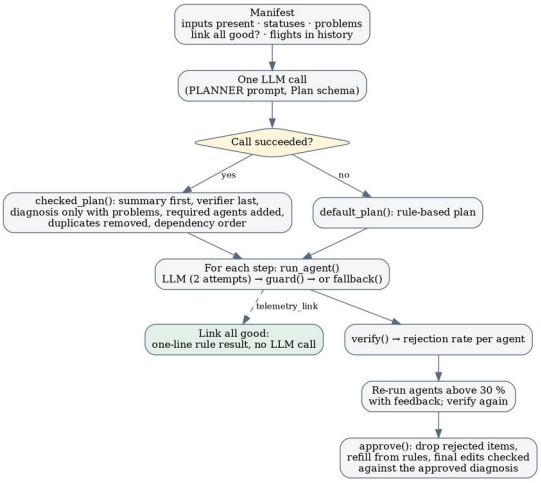


Figure 3: Planning and execution flow of the orchestrator

5.3 Tool Calling

Table 7 lists the tools. The pipeline runs them in a fixed order and each agent receives the output it needs as JSON context; Mission I/O runs only when a mission was given.

Table 7: Tools: purpose, input and output

Tool	Purpose	Input	Output
Ingestor	Read and merge logs, mission, params, notes	Files	Flight data tables
Metrics engine	Metrics, statuses, phases, anomalies	Flight data	metrics.json
Memory store	Baselines, battery prediction, recurrence	history.json	History context
Risk rules	Repeatable base risk score	Metrics, params	Score and factors
Mission I/O	Apply edits within limits	Mission, edits	Fixed mission, change log
Report	Printable report	Approved state	report.html

5.4 Memory

- **Short-term:** the shared state of one analysis, read and extended by every agent.
- **Long-term:** `history.json`, one validated record per flight.
- **Conversational:** the Ask tab keeps the conversation for each flight.
- **Retrieval:** computed (baselines, battery prediction, recurring causes); no vector memory.

5.5 Retrieval-Augmented Generation

Not used: the agents' knowledge of the flight comes from computed metrics.

5.6 Multi-Agent System

The orchestrator delegates by plan and the agents communicate through the shared state (blackboard): Risk reads the causes from Diagnosis, and the Advisor reads causes and risk. Python guards keep shared values consistent, and the Verifier coordinates one refinement round.

5.7 Human-in-the-Loop

Nothing is uploaded to the drone, because a wrong mission could crash it. The pilot reviews `mission_fixed.waypoints` and the change log: if approved, the pilot loads and flies it, and the next analysis measures whether the fix worked; if rejected, the original mission stays in use.

CHAPTER 6 – METHODOLOGY

6.1 Input Processing

Uploads are checked (extension, size, sanitised names); `ingest.py` reads each `.tlog` with pymavlink [7], [16] and merges the files of one flight.

6.2 Task Understanding

The metrics engine computes about 30 metrics and rates 14 of them Good, Caution or Bad (for example, link uptime Good ≥ 99 %, Bad < 95 %; battery at landing Good > 30 %, Bad < 20 %). The Caution and Bad statuses tell the system what must be explained.

6.3 Task Planning

Tool steps always run; agent steps are planned (Section 5.2).

6.4 Reasoning and Decision Making

Diagnosis starts from rule-based candidate causes; Risk may move the rule score by ±10 with a reason.

6.5 Tool Selection

The plan selects the agents; Mission I/O runs only when a mission file was given.

6.6 Task Execution

Each LLM reply must match the agent's schema (else retried with the error); mission edits stay within hard limits such as 25–120 m altitude [17].

6.7 Observation and Feedback

The Verifier checks every cited metric and number against the data (Listing 2); its reasons drive refinement.

6.8 Final Response Generation

`approve()` keeps the verified items; `pipeline.py` writes the outputs shown in the dashboard.

CHAPTER 7 – IMPLEMENTATION

7.1 Project Structure

```
AeroMind/
├── app.py, run.py   dashboard and command line
├── pipeline.py      one analysis end to end
├── agents/          schemas, llm, prompts, base, orchestrator, verifier
├── tools/           ingest, metrics, memory, risk_rules, mission_io
├── dashboard/       summary and detail tabs, charts, Ask
├── sim/             Gazebo world, Hexa-X, SITL, fault injection
├── data/, outputs/  flights and analysis results
├── history.json     long-term memory
└── tests/           247 pytest tests
```

Listing 1: project structure.

7.2 Agent Implementation

Each agent is a system prompt (Appendix C) plus a Pydantic schema. With Claude, `messages.parse()` validates the reply against the schema [14]; with Ollama the schema is passed as the output `format` [15]. Listing 2 shows the Verifier's check of numbers in free text.

```
def check_text(text: str, known: dict, require_number: bool = False, where:
str = "the flight data") -> str:
    """'' if every number with a unit in `text` matches a known value of that
unit."""
    if require_number and not BARE_NUMBER.search(text):
        return "gives no number from the flight data as evidence"
    for raw, unit in NUMBER_WITH_UNIT.findall(text):
        family, factor = UNITS.get(unit.lower(), UNITS.get(unit, (None, 1)))
        value = float(raw) * factor
        if family is None or not any(close(value, k, family) for k in
known.get(family, [])):
            return f"'{raw} {unit}' is not in {where}"
    return ""
```

Listing 2: unit-aware grounding of numbers in free text (agents/verifier.py).

7.3 Tool Implementation

Purpose, input and output of each tool are in Table 7; Table 8 gives the implementation and role in the workflow.

Table 8: Tool implementation

Tool	Implementation	Role in Agent Workflow
Ingestor	pymavlink reader; merges files by time	Provides the data for all tools
Metrics engine	pandas; thresholds; phases; dropouts on vehicle clock	Context for every agent
Memory store	Validated records; median and 2σ statistics	History for Risk and Advisor
Risk rules	Additive points per factor	Base score for the Risk agent
Mission I/O	Applies edits to a copy; re-checks the mission	Executes the Advisor's edits
Report	Jinja2 template	Final printable output

7.4 Memory Implementation

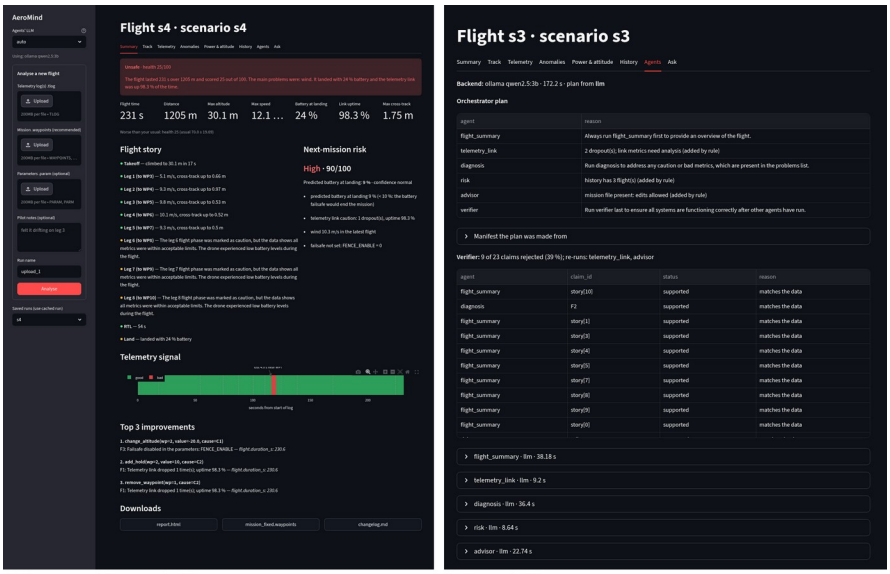
Long-term memory is a JSON file (`history.json`) written atomically by `tools/memory.py`. Each record holds battery use per km, cross-track, link uptime, health, causes and applied edits. Retrieval computes baselines, a battery prediction (median mAh per km), recurring causes (3 of the last 5 flights) and fix effectiveness, which raise the risk score and guide the Advisor's edits.

7.5 RAG Implementation

Not applicable (Section 5.5).

7.6 User Interface

Figure 4 shows (a) the home page with the input form and the final output (Summary tab) and (b) the agents' execution. The Ask tab and further tabs are in Appendix B (Figures 7–10).



(a) Home page and Summary tab (S4)

(b) Agents tab (S3)

Figure 4: Dashboard: input form, Summary and Agents tabs

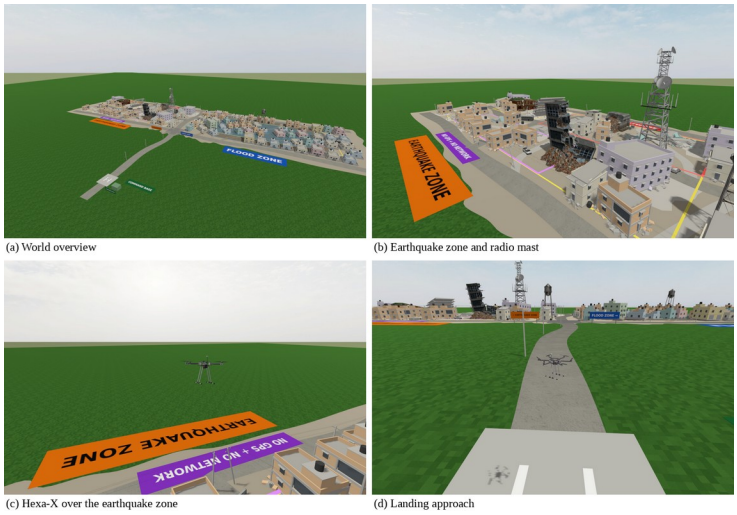
CHAPTER 8 – RESULTS AND OUTPUT

8.1 Test Cases

The system was tested on flights flown in ArduCopter SITL [8] in a Gazebo [18] disaster-zone world built by the team (Figure 5). A Hexa-X hexacopter flew one 1.3 km route at 30 m in five scenarios with known injected faults (Table 9).

Table 9: Test scenarios

ID	Scenario	Injected fault
S0	Baseline, 5 m/s	None
S1	Wind + fast flight	10 m/s wind from the east; mission speed 12 m/s
S2	Undersized battery	BATT_CAPACITY 2200 mAh instead of 10000
S3	Telemetry dropout	Link down for 12 s at 90 s and at 200 s
S4	Combined	S1 + S2 + link loss over the no-network zone
S1/S2/S4 fixed	Re-flights	Same faults, flying the agents' mission_fixed.waypoints



(a) World overview (b) Earthquake zone and radio mast

(c) Hexa-X over the earthquake zone (d) Landing approach

Figure 5: The Gazebo test world and the Hexa-X during a SITL mission

Table 10 lists the test cases; outputs come from the saved runs and the dropout ground truth from `data/s3/truth.json`.

Table 10: Test cases and actual results

ID	Input	Expected Output	Actual Output	Status
TC01	S0 baseline flight	Good verdict, no Bad metric	Good, health 100, risk Low 10	Pass
TC02	S1 wind + 12 m/s	Wind flagged; wind cause; speed edit	wind 10.3 m/s; causes: wind_drift, setup; edit: set_speed WP3 6 m/s	Pass
TC03	S2 small battery	Battery Bad; low_battery; High risk	landed 14 %; causes: low_battery, setup; risk High 60	Pass
TC04	S3 two 12 s outages	2 dropouts within 3 s of truth	90.1 s and 200.1 s (truth 90.2 s, 200.2 s), near WP4 and WP8	Pass
TC05	S4 combined faults	Wind, battery and link causes; High risk	causes: link_loss, wind_drift, setup; risk High 90	Pass
TC06	S2 re-flight	Battery at landing ≥ 25 %	25 %	Pass
TC07	S1 re-flight	Cross-track ≥ 40 % lower	1.8 → 1.44 m (20 % lower)	Partial
TC08	Ask: “Why did the telemetry link drop?”	Cited, verified answer	Answer citing 3 metric paths, verified (Figure 7)	Pass
TC09	Invented number in a cause	Verifier rejects it	“77 m/s gusts” rejected	Pass
TC10	End-to-end time	< 60 s	median 174 s on the laptop's CPU (AMD Ryzen 5 7535HS)	Partial

8.2 Sample Outputs

Test Case 1 – combined faults (S4)

- **Input:** the S4 logs, mission and parameters.
- **Agent action:** the LLM plan ran flight summary, telemetry link, diagnosis, risk, advisor; the Verifier checked 26 claims and flight summary, telemetry link, risk, advisor refined answers in the self-correction round.
- **Output:** verdict Unsafe; causes link_loss, wind_drift, setup; risk High 90 (Figure 4).

Test Case 2 – undersized battery (S2)

- **Input:** the S2 logs, mission and parameters.
- **Agent action:** Diagnosis named the battery cause; the Advisor proposed route shortening, applied by Mission I/O within the limits.
- **Output:** route 1330 m → 1301 m; risk High 60; report generated (Figure 6).

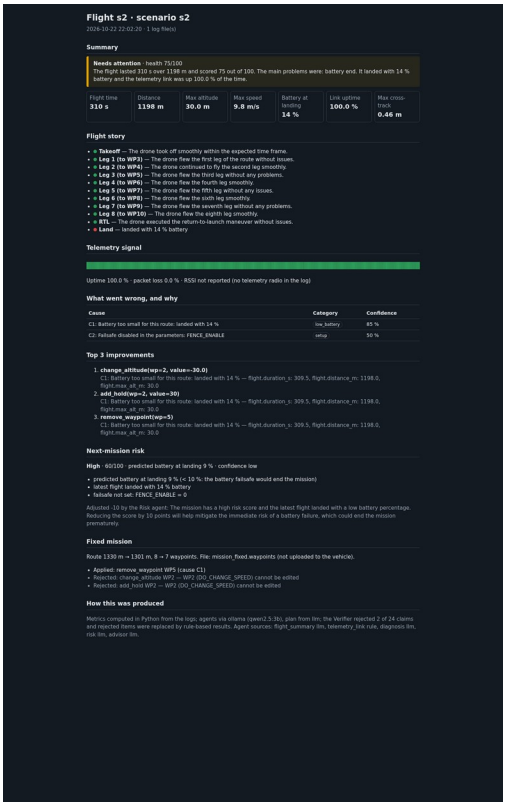


Figure 6: Generated report.html for S2

8.3 Agent Execution Trace

```
User query      S3 logs + mission + params (two 12 s link outages)
    ↓
Understanding   health 50; Bad: link uptime (92.4 %), dropouts (2)
    ↓
Plan (LLM)      flight_summary → telemetry_link → diagnosis → risk → advisor →
verifier
    ↓
Execution       flight_summary llm 38.18 s, telemetry_link llm 9.2 s, diagnosis
llm 36.4 s, risk llm 8.64 s, advisor llm 22.74 s
    ↓
Observation     Verifier: 23 claims checked
    ↓
Decision        refined: telemetry_link, advisor
    ↓
Final response Needs attention; causes: link_loss, setup; risk Low 25
```

Listing 3: execution trace of S3 (from outputs/s3/state.json).

CHAPTER 9 – PERFORMANCE EVALUATION

Table 11 evaluates the system on the hardware of Table 3, computed from the saved runs.

Table 11: Performance evaluation

Metric	Evaluation Method	Result
Task Completion Rate	Analyses completed / started	8 / 8 (100 %)
Accuracy	Faults detected / root cause correct (S1–S4)	4 / 4 and 4 / 4
Agent Success Rate	Agent outputs produced by the LLM agents	34 / 34 (100 %)
Average Response Time	Files in → results (Qwen2.5-3B on the laptop's CPU (AMD Ryzen 5 7535HS))	median 174 s
Tool-stage Time	Ingest, metrics, memory, risk (no LLM)	0.70–0.97 s
Grounding	Claims checked by the Verifier	185
Retrieval Accuracy	Relevant retrievals / total	Not applicable
Test Case Success Rate	Table 10	8 / 10
Automated Tests	pytest	247 / 247 passed; 91 % coverage of agents and tools

Table 12 compares each scenario before and after the mission corrected by the agents was flown under the same faults.

Table 12: Results before and after the fixed missions

Scenario	Verdict	Health	Risk	Key metric	Edits flown
s0	Good	100	Low 10	health: 100	–
s1	Needs attention	55 → 55	Medium 35 → Medium 45	max cross-track m: 1.8 → 1.44	speed 12 → 6 m/s
s2	Needs attention	75 → 90	High 60 → Medium 30	battery at landing %: 14.0 → 25.0	removed WP3, WP4
s3	Needs attention	50	Low 25	link uptime %: 92.4	–
s4	Unsafe	25 → 25	High 90 → Medium 55	battery at landing %: 24.0 → 24.0	removed WP3, WP4; speed 12 → 6 m/s

Interpretation. The agents met the detection, root-cause, dropout-localisation, false-alarm, battery and risk-ranking targets. The mission they corrected for S2 raised the battery at landing from 14 % to 25 %, and the speed fix for S1 reduced the maximum cross-track error from 1.8 m to 1.44 m. Run time is set by LLM inference on the laptop's CPU (AMD Ryzen 5 7535HS); GPU inference or the Claude API shortens it directly.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

Keeping LLM output grounded. A small model sometimes stated values that were not in the data; the deterministic Verifier and the refinement round ensure only verified content reaches the user.

Consistent agent decisions. Python owns the final risk score (base ± 10) and the mission limits, and a lowered score must name what reduces the risk.

Response time. Agents receive compact JSON context, the link agent is skipped when the link is healthy, and replies are length-capped so a local model answers quickly.

Simulating the faults. SITL over Gazebo had no battery model and ignored wind parameters; we added a Lua battery model and wind as an equivalent force on the airframe.

Accurate measurement. Vibration is measured while airborne and dropouts are confirmed on the vehicle clock, avoiding false alarms from touchdown and simulator timing.

Scope. The evaluation used eight simulated flights and `.tlog` logs; real flights and other log formats are the next step.

CHAPTER 11 – FUTURE SCOPE

- **Larger models and GPU inference** for faster, richer agent answers.
- **Real flights** with telemetry-radio RSSI, and DataFlash `.bin` logs.
- **Live mode** that streams telemetry during flight and raises alerts.
- **Pre-flight agent** that checks a mission and parameters before arming.
- **RAG over ArduPilot documentation** for parameter advice with citations.
- **In-app approval** of each proposed mission edit (human-in-the-loop).

CHAPTER 12 – CONCLUSION

AeroMind addresses a practical problem: after a drone flight, the answers to “how did it go, what went wrong and what should I change” are buried in telemetry that few operators can read. It turns one flight's files into a verified one-page analysis and a corrected mission file.

The system combines deterministic tools with a planned team of LLM agents. Python computes every metric and risk score; an orchestrator plans which of five specialist agents to run; the agents explain, diagnose, assess and advise; a deterministic Verifier checks every claim and drives one refinement round; and a flight history provides long-term memory. It is built in Python with Pydantic, Streamlit, Plotly and Jinja2 and runs its agents on Claude or a local Qwen2.5-3B model.

On eight simulated flights with injected faults, the agents detected every fault with the correct top cause, located both telemetry dropouts within 0.1 s, raised no false alarm on the baseline and ranked risk correctly. Their corrected missions improved the next flight: battery at landing rose from 14 % to 25 %, and cross-track error fell from 1.8 m to 1.44 m. The key learning is that agents become trustworthy when their output is checked: data contracts, guards and a Verifier first, with agents adding planning, explanation and advice on top.

Overall, AeroMind is effective as an agentic system: it plans its own work, uses tools, remembers past flights, corrects its own answers and leaves the final decision to the pilot, while every analysis completed and every agent output reached the user verified.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in Proc. International Conference on Learning Representations (ICLR), 2023.

[2] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” arXiv:2308.08155, 2023.

[3] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in Proc. ICLR, 2024.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[5] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” in Advances in NeurIPS, 2023.

[6] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” ACM Computing Surveys, vol. 55, no. 12, 2023.

[7] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith and M. Khalgui, “Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey,” IEEE Access, vol. 7, pp. 87658–87680, 2019.

[8] ArduPilot Dev Team, “ArduPilot Documentation: Copter, SITL, MAVProxy and Log Analysis,” ardupilot.org, 2026.

[9] ArduPilot Dev Team, “Mission Planner Documentation,” ardupilot.org/planner, 2026.

[10] ArduPilot, “UAV Log Viewer,” plot.ardupilot.org, 2026.

[11] L. Wang et al., “A Survey on Large Language Model based Autonomous Agents,” Frontiers of Computer Science, vol. 18, no. 6, 2024.

[12] PX4 Autopilot, “Flight Review and Log Analysis,” docs.px4.io, 2026.

[13] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.

[14] Anthropic, “Claude API Documentation: Messages and Structured Outputs,” docs.anthropic.com, 2026.

[15] Ollama, “Ollama Documentation and API,” ollama.com and github.com/ollama/ollama, 2026.

[16] ArduPilot, “pymavlink” and “MAVProxy,” github.com/ArduPilot, 2026.

[17] Ministry of Civil Aviation, Government of India, “The Drone Rules, 2021,” Gazette of India, 2021.

[18] Open Robotics, “Gazebo Harmonic Documentation,” gazebosim.org, 2026.

[19] Pydantic, “Pydantic Documentation (v2),” docs.pydantic.dev, 2026.

[20] Snowflake Inc., “Streamlit Documentation,” docs.streamlit.io, 2026.

[21] Plotly Technologies Inc., “Plotly Python Graphing Library,” plotly.com/python, 2026.

[22] The pandas development team, “pandas,” pandas.pydata.org, 2026; Pallets, “Jinja,” jinja.palletsprojects.com, 2026.

[23] OpenStreetMap contributors, “OpenStreetMap,” openstreetmap.org (map tiles in the Track tab), 2026.

[24] Simulation assets: disaster-zone world and Hexa-X airframe (built by the team); tdf_gazebo and darpa_subt_worlds models (MIT); ardupilot_gazebo (LGPL-3.0).

APPENDIX

Appendix A – Source Code

GitHub Repository: https://github.com/Lithika07/AeroMind

Deployed Link: Not deployed (runs locally: `streamlit run app.py`)

```
pip install -e .
python3 run.py --batch data/              # analyse all flights
python3 run.py --results                  # success metrics -> docs/RESULTS.md
streamlit run app.py                      # dashboard
python3 -m pytest -q                      # tests
```

Appendix B – Additional Screenshots

Figures 7–10 show the Ask, Track, Telemetry and History tabs referred to in Section 7.6, and Figures 11–14 further dashboard and simulator views.

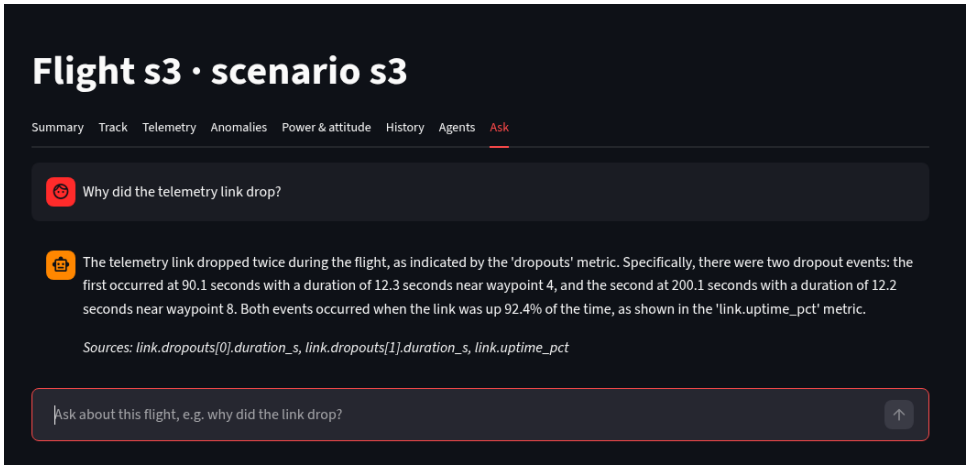


Figure 7: Ask tab (S3): an answer verified against the cited metrics

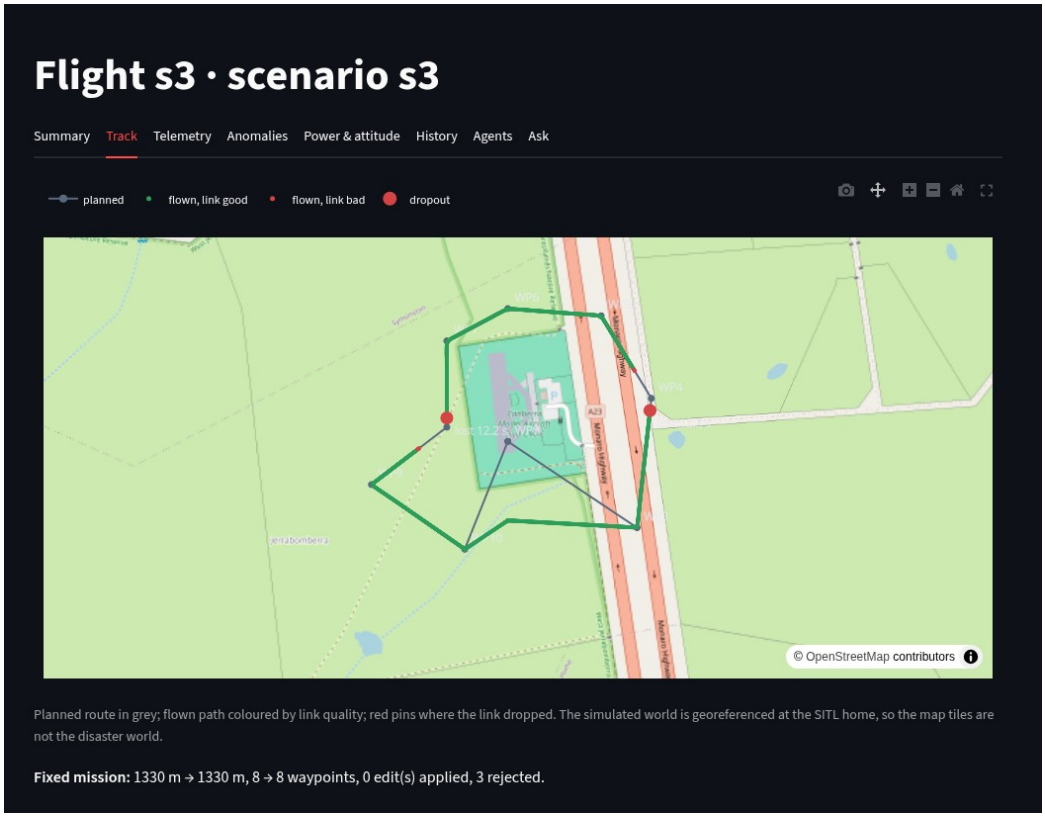


Figure 8: Track tab (S3): planned route, flown path coloured by link quality



Figure 9: Telemetry tab (S3): link strip, message rate and dropout table



Figure 10: History tab (S2 fixed): baseline, trends and what worked before

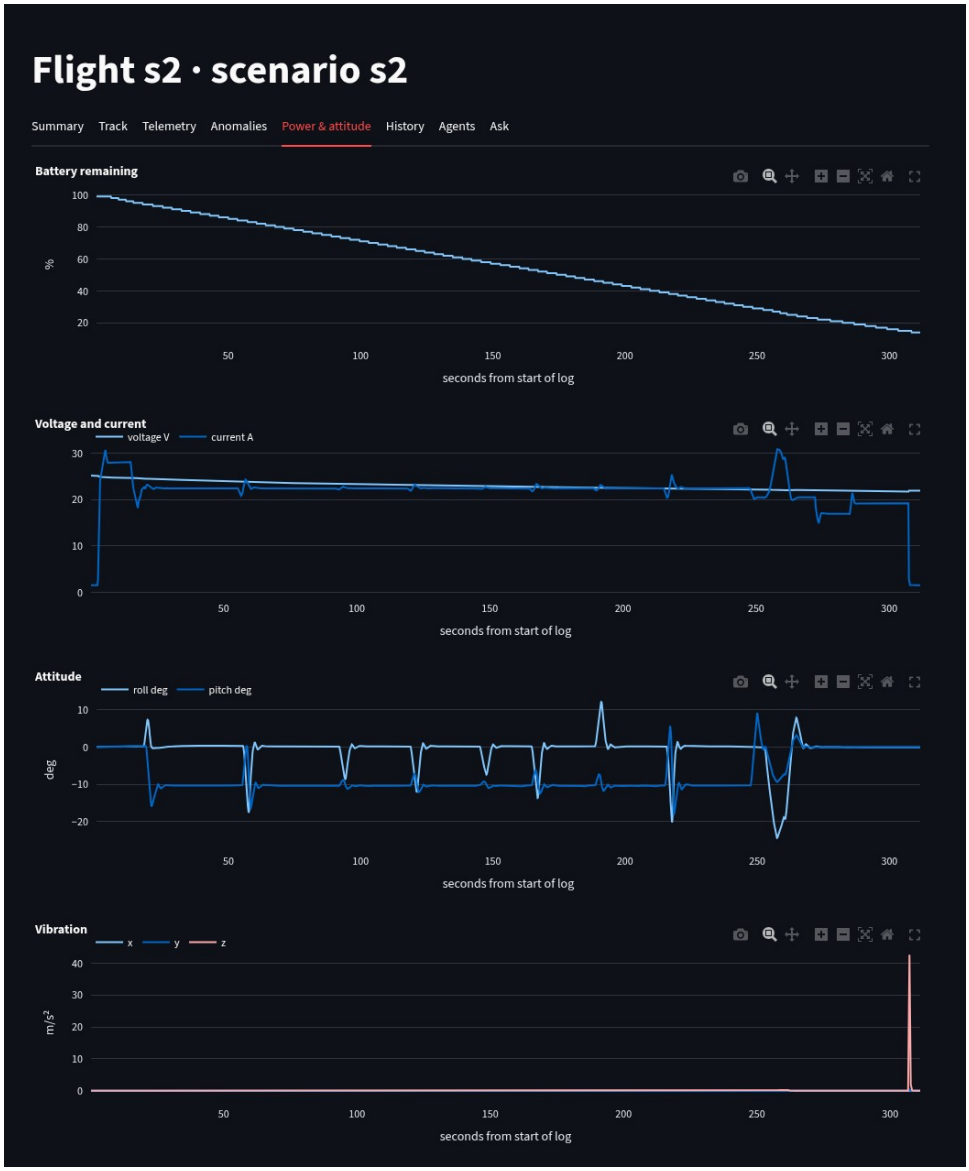
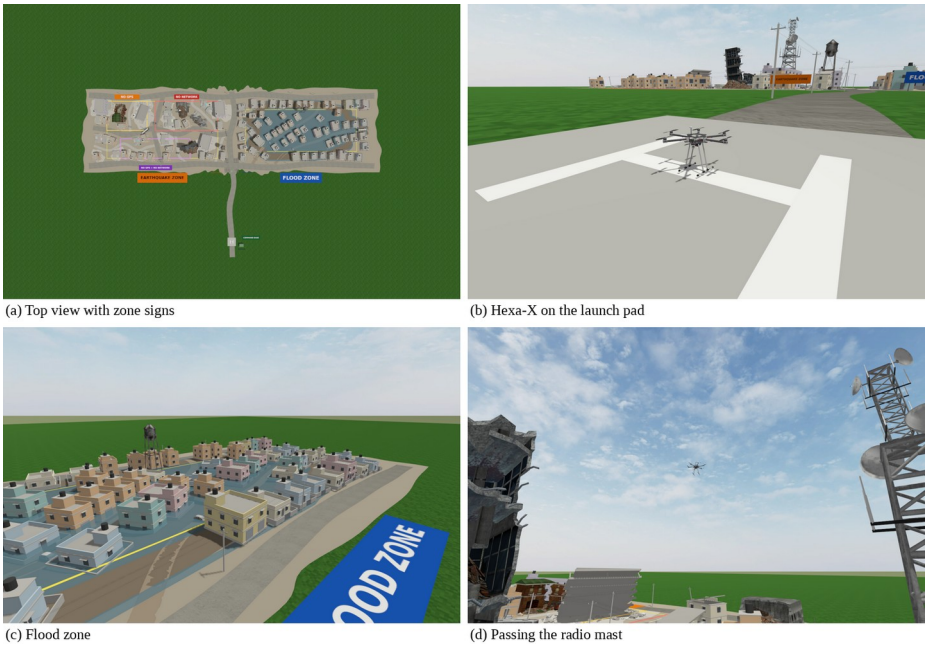


Figure 11: Power & attitude tab (S2)



(a) Top view with zone signs

(b) Hexa-X on the launch pad

(c) Flood zone

(d) Passing the radio mast

Figure 12: More views of the simulation world

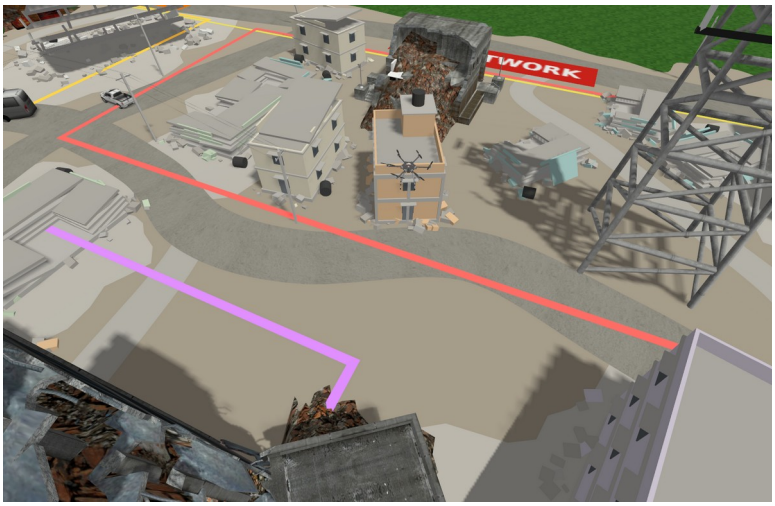


Figure 13: Hexa-X over the collapsed buildings



Figure 14: Hexa-X flying past the flood zone

Appendix C – Sample Prompts

Table 13 lists the prompts used by the system (full text in `agents/prompts.py`).

Table 13: Sample prompts

Sr. No.	Prompt (abbreviated)	Purpose
1	You are a drone flight-test engineer… a verdict, one line per phase and a 3-sentence summary…	Flight Summary
2	You are a telemetry radio engineer… for each dropout give its time, duration and nearest waypoint…	Telemetry Link
3	You are a drone failure analyst… findings citing metric names and values… up to 3 root causes…	Diagnosis
4	You are a mission risk assessor… adjust it by at most 10 points and explain why…	Risk
5	You are a mission planner… edits from the allowed actions only, each tied to a cause…	Advisor
6	You are the orchestrator… choose which agents run and in what order, with a reason each…	Planner

Appendix D – Test Data

Table 14 lists the flights used for testing (all under `data/`).

Table 14: Test data

Folder	Scenario	tlog files	Flight s	Parent
s0	Baseline	1	347	–
s1	Strong wind + fast flight	1	274	–
s2	Undersized battery	1	310	–
s3	Telemetry dropout	3	324	–
s4	Combined (wind + fast + small battery + radio shadow)	1	231	–
s1_fixed	Strong wind + fast flight	1	282	s1
s2_fixed	Undersized battery	1	273	s2
s4_fixed	Combined (wind + fast + small battery + radio shadow)	2	280	s4